

UNIVERSITÉ PROTESTANTE AU CONGO
FACULTÉ DES SCIENCES INFORMATIQUES
LICENCE 2 - SYSTÈME LMD



MONOGRAPHIE

**CONCEPTION ET DÉVELOPPEMENT DE SMART
FACULTY**

Une plateforme numérique pour simplifier la gestion académique d'une faculté

Travail réalisé par

YAMBO NGOLU GRACE

Matricule : SI023724

Kinshasa, 2026

Résumé

Cette monographie présente Smart Faculty, une plateforme numérique conçue pour une faculté universitaire. Son but est de rassembler dans une seule application les informations qui sont souvent dispersées : comptes, cours, inscriptions, enrôlements, notes, résultats, annonces, réclamations, projets et présences. L'application utilise Flutter pour afficher les écrans, FastAPI pour appliquer les règles et MySQL pour conserver les données. Le travail a suivi une démarche simple : comprendre les besoins, organiser les informations, développer les fonctions une par une et vérifier leur fonctionnement à l'aide de tests. La version décrite permet déjà de gérer plusieurs opérations académiques et de limiter les actions selon le rôle de chaque utilisateur. La reconnaissance faciale est préparée, mais son utilisation complète dans un environnement réel doit encore être finalisée et évaluée. Le projet montre qu'un outil bien organisé peut réduire les tâches manuelles, rendre les informations plus faciles à retrouver et améliorer le suivi des activités d'une faculté.

Mots-clés : gestion académique, plateforme numérique, base de données, sécurité, rôles, présences, tests.

Table des matières

Résumé	1
Table des matières.....	2
1. Introduction générale.....	3
1.1 Contexte.....	3
1.2 Problématique	3
1.3 Objectifs	4
1.3.1 Objectif général	4
1.3.2 Objectifs spécifiques.....	4
1.4 Méthodologie adoptée	4
1.5 Structure du document.....	4
2. Cadre théorique et technologique	5
2.1 Définitions des concepts clés	5
2.2 Présentation des outils ou technologies utilisés.....	7
3. Réalisation du projet	8
3.1 Analyse des besoins.....	8
3.2 Architecture de la solution	10
3.3 Étapes de mise en œuvre.....	11
3.4 Résultats obtenus.....	16
4. Difficultés rencontrées et limites	17
4.1 Difficultés rencontrées.....	17
4.2 Limites de la solution	18
5. Conclusion générale	19
5.1 Récapitulatif.....	19
5.2 Apports du projet.....	19
5.3 Perspectives	19
6. Bibliographie et Webographie	20
7. Annexes	21

1. Introduction générale

1.1 Contexte

Dans une faculté, beaucoup d'informations circulent chaque jour. Les étudiants cherchent leurs notes, leurs cours ou les annonces. Les enseignants doivent communiquer des consignes, saisir des notes et suivre leurs étudiants. L'apparitorat organise les inscriptions, les enrôlements et plusieurs dossiers académiques. Les autorités ont besoin d'une vue claire pour prendre de bonnes décisions.

Lorsque ces informations sont gardées dans des cahiers, des feuilles séparées, des groupes de messagerie ou plusieurs fichiers, le travail devient lent. Une même information peut être répétée, perdue ou difficile à vérifier. Smart Faculty est né de ce constat. Le projet propose un seul espace numérique dans lequel chaque personne voit uniquement ce qui concerne son travail.

Smart Faculty est destiné à une faculté et non à toute une université. Ce choix rend le projet plus réaliste pour un travail de deuxième année. Il permet de commencer par un environnement limité, de comprendre ses règles, puis d'améliorer progressivement la solution.

1.2 Problématique

Le problème principal peut être formulé ainsi : comment créer une plateforme simple et sécurisée qui rassemble les principales activités académiques d'une faculté, donne à chaque utilisateur un espace adapté à son rôle et conserve un historique clair des opérations ?

Cette question en contient plusieurs autres : comment éviter qu'un étudiant voie les données d'un autre étudiant ? Comment permettre à un enseignant d'agir seulement sur ses cours ? Comment suivre une réclamation sans la perdre ? Comment enregistrer une présence de manière fiable ? Comment faire évoluer la base de données sans détruire les informations déjà enregistrées ?

1.3 Objectifs

1.3.1 Objectif général

L'objectif général est de concevoir et de développer une plateforme numérique capable de simplifier la gestion académique d'une faculté, tout en protégeant les informations et en respectant les responsabilités de chaque utilisateur.

1.3.2 Objectifs spécifiques

- Créer une connexion sécurisée par adresse électronique et mot de passe.
- Gérer les utilisateurs et leurs rôles : étudiant, enseignant, chef de promotion, surveillant, appariteur, doyen, vice-doyen et administrateur.
- Organiser les années académiques, les semestres, les promotions, les cours et les inscriptions.
- Permettre la saisie, la publication et la consultation contrôlée des notes et des résultats.
- Centraliser les annonces dans une Valve numérique liée aux cours.
- Suivre les demandes d'inscription, les enrôlements, les réclamations et les projets académiques.
- Préparer un système de présences pouvant évoluer vers la reconnaissance faciale.
- Vérifier le fonctionnement du projet par des tests répétés sur une base séparée de la base principale.

1.4 Méthodologie adoptée

Le travail a avancé par petites étapes. D'abord, les besoins des utilisateurs ont été étudiés. Ensuite, les écrans, les rôles et les informations à conserver ont été organisés. Après cela, le serveur, la base de données et l'application Flutter ont été développés module par module. Enfin, des tests ont été exécutés pour vérifier les règles, détecter les erreurs et éviter qu'une nouvelle modification casse une fonction déjà terminée.

1.5 Structure du document

Le deuxième chapitre explique les notions et les outils utilisés. Le troisième présente la réalisation du projet. Le quatrième expose les difficultés et les limites. Le cinquième donne le bilan et les perspectives. La bibliographie, la webographie et les annexes terminent le document.

2. Cadre théorique et technologique

2.1 Définitions des concepts clés

Système d'information académique : C'est l'ensemble formé par les personnes, les règles, les documents, les données et les logiciels utilisés pour faire fonctionner une faculté. Smart Faculty est une partie numérique de ce système.

Plateforme numérique : C'est un espace informatique qui réunit plusieurs services. Au lieu d'ouvrir plusieurs fichiers ou applications, l'utilisateur trouve ses fonctions principales au même endroit.

Interface utilisateur : C'est ce que la personne voit à l'écran : boutons, menus, formulaires, tableaux et messages. Dans Smart Faculty, l'interface est créée avec Flutter.

Base de données : On peut la comparer à une grande armoire bien rangée. Chaque tiroir contient une catégorie d'informations : étudiants, cours, notes ou présences. MySQL joue ce rôle dans le projet.

Table et relation : Une table regroupe des informations de même nature. Une relation est le lien entre deux tables. Par exemple, un cours est lié à une promotion et un étudiant est lié à ses inscriptions.

Architecture client-serveur : L'application affichée à l'utilisateur demande un service, le serveur réfléchit et répond. Flutter est le côté visible, FastAPI est la partie qui contrôle les règles et MySQL conserve les données.

API : Une interface de programmation est comme un guichet entre Flutter et FastAPI. Flutter envoie une demande précise et le serveur renvoie une réponse dans un format compréhensible par l'application.

Authentification : C'est la vérification de l'identité. Elle répond à la question : « Qui essaie de se connecter ? »

Autorisation : Elle répond à une autre question : « Cette personne a-t-elle le droit d'effectuer cette action ? » Un étudiant peut lire ses notes, mais il ne peut pas modifier les notes de toute la promotion.

Jeton de connexion : C'est un badge numérique temporaire donné après une connexion réussie. Il évite de redemander le mot de passe à chaque page et permet au serveur de reconnaître l'utilisateur.

Rôle utilisateur : Le rôle décrit la responsabilité d'une personne. Il détermine les menus visibles et les actions permises.

Migration de base de données : C'est une modification numérotée de la structure de la base. Elle permet d'ajouter une table ou un champ de manière contrôlée.

Test automatisé : C'est une vérification exécutée par l'ordinateur. Le test pose une question au programme et compare la réponse obtenue avec la réponse attendue.

Reconnaissance faciale : C'est une technique qui compare les caractéristiques d'un visage avec un profil enregistré. Dans Smart Faculty, cette partie est préparée, mais son déploiement réel doit encore être complété, sécurisé et testé.

Intelligence artificielle : C'est un ensemble de techniques qui permettent à une machine d'analyser des informations et d'aider à produire une réponse. Dans la vision future de Smart Faculty, un assistant pourra répondre à des questions académiques simples.

2.2 Présentation des outils ou technologies utilisés

Outil	Rôle dans Smart Faculty	Raison du choix
Flutter et Dart	Créer les écrans web et mobile.	Un même projet peut servir plusieurs appareils et garder une apparence cohérente.
Python	Écrire la logique du serveur.	Le langage est lisible et convient bien à un projet pédagogique évolutif.
FastAPI	Créer les services utilisés par Flutter.	Il facilite la validation des données, la documentation des routes et l'organisation du serveur.
MySQL avec InnoDB	Conserver les données.	Il gère les relations, les transactions et de grandes quantités d'informations.
SQLAlchemy	Faire le lien entre les objets Python et les tables MySQL.	Le code peut décrire clairement les données sans écrire chaque requête à la main.
Alembic	Numéroter les changements de la base.	Chaque évolution peut être appliquée et vérifiée de manière ordonnée.
Pytest et Flutter test	Exécuter les tests automatisés.	Ils aident à détecter rapidement une erreur après une modification.
Visual Studio Code	Écrire et organiser le code.	Il regroupe l'éditeur, le terminal et les outils de développement.
Git et GitHub	Conserver l'historique du projet.	Ils permettent de retrouver les changements et de travailler plus proprement.
MySQL Workbench et WAMP	Administrer et lancer MySQL en local.	Ils simplifient la consultation de la base et le travail sur un ordinateur Windows.

Ces outils ont été choisis pour séparer clairement l'affichage, les règles et le stockage. Cette séparation rend le projet plus facile à comprendre, à tester et à améliorer.

3. Réalisation du projet

Ce chapitre est le cœur du travail. Il explique ce que les utilisateurs attendent, comment la solution est organisée et comment les fonctions ont été développées.

3.1 Analyse des besoins

Utilisateur	Besoin principal
Étudiant	Consulter ses cours, annonces, notes, résultats, enrôlements, projets, réclamations et présences.
Enseignant	Voir ses cours, publier des annonces, créer des évaluations, saisir des notes et suivre les projets encadrés.
Chef de promotion	Valider certaines opérations liées à la promotion et confirmer la deuxième séance de cours.
Surveillant	Ouvrir une séance, identifier les étudiants et enregistrer leur présence ou le motif du refus.
Appariteur	Traiter les inscriptions, gérer les enrôlements, organiser les projets et suivre plusieurs activités académiques.
Doyen / Vice-doyen	Consulter les informations globales, les résultats officiels et les décisions académiques.
Administrateur	Gérer les comptes, les rôles, les paramètres techniques et la sécurité générale.

3.1.1 Besoins fonctionnels

Un besoin fonctionnel décrit ce que le système doit faire.

Fonction	Attente
Connexion et inscription	Permettre la connexion, la demande d'inscription et la validation par un responsable.
Gestion académique	Gérer les années, semestres, promotions, cours, enseignants et étudiants.
Notes et résultats	Créer des évaluations, enregistrer les notes, calculer les résultats et publier uniquement les données autorisées.
Valve	Publier des annonces, consignes et documents liés à un cours.
Réclamations	Créer, assigner, suivre, répondre et clôturer une réclamation.
Enrôlement	Enregistrer et valider l'enrôlement, puis produire une fiche consultable par l'étudiant.
Projets	Créer un projet et affecter un encadreur selon sa spécialité.
Présences	Créer les séances, vérifier les conditions d'accès, enregistrer les présences et garder les corrections.
Décisions	Conserver les membres du jury, les décisions et les résultats officiels du semestre.

3.1.2 Besoins non fonctionnels

- Simplicité : les écrans doivent rester compréhensibles, même pour une personne peu habituée aux outils numériques.
- Sécurité : le serveur doit vérifier l'identité et les droits avant chaque action sensible.
- Fiabilité : les opérations importantes doivent être testées et les erreurs clairement signalées.
- Traçabilité : les décisions, corrections et changements importants doivent laisser une trace.
- Évolutivité : de nouveaux modules doivent pouvoir être ajoutés sans réécrire tout le projet.

3.2 Architecture de la solution

Smart Faculty utilise une architecture en trois parties. Cette organisation peut être comparée à un restaurant : Flutter est la salle où le client passe sa commande, FastAPI est la cuisine qui vérifie et prépare la réponse, et MySQL est la réserve où les informations sont rangées.

Architecture simple de Smart Faculty

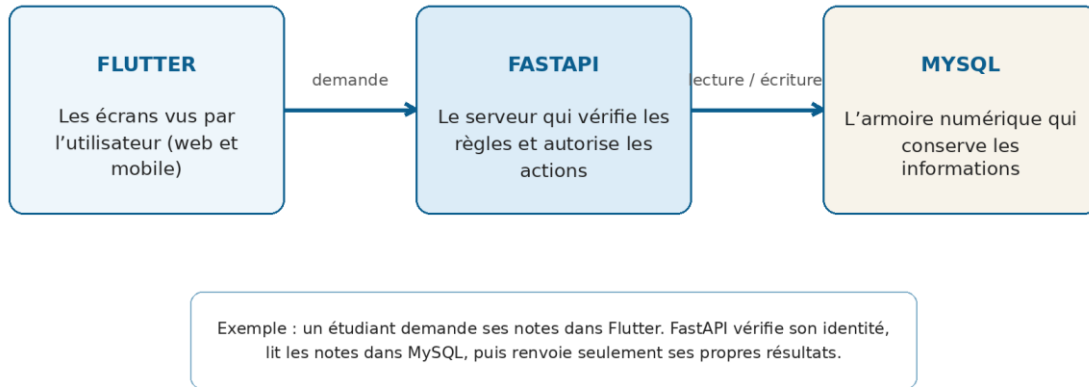


Figure 1. Architecture simplifiée de Smart Faculty.

Une demande suit donc un trajet simple : l'utilisateur clique dans Flutter, FastAPI reçoit la demande, vérifie le jeton de connexion et le rôle, applique la règle métier, consulte ou modifie MySQL, puis renvoie une réponse. Cette séparation évite que l'application affiche directement la base de données et renforce le contrôle des accès.

3.3 Étapes de mise en œuvre

3.3.1 Phase de conception

La première étape a consisté à définir clairement le périmètre : Smart Faculty gère une faculté. Les utilisateurs, leurs responsabilités et les principales règles ont ensuite été décrits dans des documents d'analyse.

1. Identifier les acteurs et ce que chacun doit pouvoir faire.
2. Découper le projet en modules : comptes, gestion académique, notes, Valve, réclamations, enrôlements, projets, présences et décisions.
3. Dessiner les écrans principaux et organiser la navigation Flutter.
4. Concevoir la base de données avec la démarche MERISE : comprendre les informations, définir leurs liens, puis les transformer en tables MySQL.
5. Prévoir une architecture modulaire pour éviter qu'un changement dans un module casse tout le projet.

3.3.2 Organisation du code

Le projet a été séparé en trois grands dossiers. Le dossier frontend contient l'application Flutter. Le dossier backend contient le serveur FastAPI, les modèles, les routes et les règles. Le dossier docs conserve la documentation. Les anciens essais en PHP et Flask ont été rangés dans un dossier d'archives appelé legacy afin qu'ils ne soient pas confondus avec la version active.

3.3.3 Construction de la base

Les données ont été regroupées par familles : identité et sécurité, organisation académique, évaluations, réclamations, délibérations, projets, Valve, présences et biométrie. Les changements de structure sont numérotés avec Alembic. Une base séparée, `smart_faculty_test`, sert aux essais afin de ne pas modifier la base principale pendant les tests.

3.3.4 Authentification et gestion des rôles

Lorsqu'une personne se connecte, le serveur vérifie l'adresse électronique et le mot de passe. Le mot de passe n'est pas conservé en clair : il est transformé en une valeur protégée. Après une connexion réussie, le serveur délivre un jeton, comparable à un badge temporaire.

Chaque demande importante doit présenter ce badge. Le serveur lit alors le rôle actif et décide si l'action est autorisée. Cette vérification est faite côté serveur, car cacher un bouton dans l'écran ne suffit pas à sécuriser une application.

Exemple d'action	Réponse attendue
Étudiant consulte ses propres notes	Autorisé
Étudiant modifie une note	Refusé
Enseignant saisit une note dans son cours	Autorisé
Enseignant agit sur un cours qui ne lui est pas attribué	Refusé
Appariteur valide un enrôlement	Autorisé
Utilisateur non connecté consulte des données privées	Refusé

3.3.5 Demandes d'inscription

Une personne qui ne possède pas encore de compte peut envoyer une demande. La demande reçoit une référence et un statut. Un responsable peut l'approuver ou la rejeter avec un motif. Cette méthode évite de créer automatiquement des comptes sans contrôle.

3.3.6 Gestion académique, notes et Valve

Les années académiques, semestres, promotions et cours forment la base de l'organisation. Un cours est rattaché à une promotion et à un semestre. Un ou plusieurs enseignants peuvent y être affectés.

Pour les notes, l'enseignant crée d'abord une évaluation, par exemple un travail pratique ou un examen. Il indique la note maximale et la pondération. Les notes peuvent être préparées avant publication. Une fois qu'une évaluation devient officielle, les modifications doivent être limitées et tracées.

La Valve est le tableau d'affichage numérique. L'enseignant peut y publier une annonce, un devoir, une consigne ou un document. L'étudiant voit seulement les publications réellement publiées et liées à ses cours.

3.3.7 Réclamations et résultats officiels

Lorsqu'un étudiant constate un problème, il peut ouvrir une réclamation en décrivant clairement la situation. La réclamation possède un statut, une priorité, des messages et un historique. Elle peut être assignée à la personne compétente, traitée puis clôturée.

Les résultats calculés par le système ne remplacent pas automatiquement le jury. Le projet prévoit une session de délibération, des membres du jury, une décision et une version officielle du résultat. Cette séparation évite de présenter un simple calcul comme une décision académique définitive.

3.3.8 Enrôlements et projets académiques

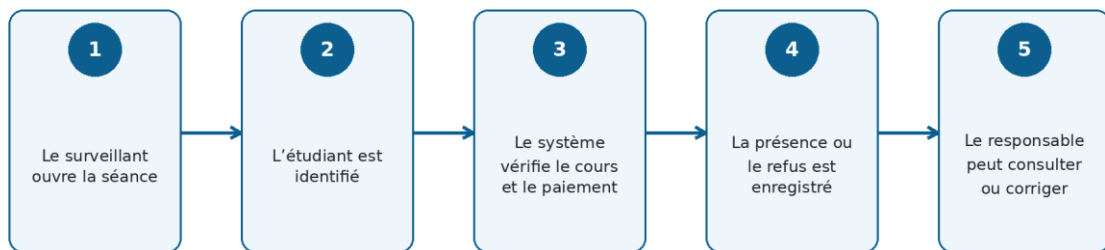
L'appareteur peut enregistrer un enrôlement pour une année académique et une promotion. L'enrôlement reçoit une référence unique et peut être validé, annulé ou consulté. L'étudiant peut ensuite accéder à sa fiche d'enrôlement.

Pour les projets, l'appareteur crée le dossier et affecte un enseignant selon le type de projet : logiciel, réseau, intelligence artificielle, système embarqué ou autre spécialité définie. L'étudiant voit son encadreur et l'enseignant voit les étudiants qui lui sont confiés.

3.3.9 Présences et préparation biométrique

Une journée peut contenir une première séance de 8 heures à 12 heures et une seconde de 13 heures à 17 heures. La deuxième séance doit être confirmée selon les règles de la faculté. Lors de l'identification, le système vérifie le cours, l'étudiant et le pourcentage de paiement observé. Si le seuil de 50 % n'est pas atteint, l'accès peut être refusé avec un motif enregistré.

Exemple : contrôle d'une présence



La règle prévue dans le projet autorise l'accès lorsque un paiement d'au moins 50 % est constaté. La deuxième séance de la journée doit aussi être confirmée par le chef de promotion.

Figure 2. Déroulement simplifié du contrôle de présence.

La structure permettant de conserver les profils biométriques et les encodages faciaux est préparée. Toutefois, l'identification faciale réelle, l'anti-usurpation, le consentement, les mesures de précision et le déploiement sur caméra doivent encore être finalisés avant toute utilisation officielle.

3.3.10 Phase de tests et contrôle de qualité

Les tests ne servent pas seulement à montrer que le programme marche une fois. Ils servent surtout à vérifier qu'une fonction continue de marcher après plusieurs modifications.

Vérification	But
Tests du backend	Vérifier les routes, les permissions, les calculs, les statuts et les règles métier.
Tests Flutter	Vérifier les services qui appellent le serveur, la navigation et certains comportements des écrans.
Analyse Flutter	Rechercher les erreurs et avertissements dans le code avant la compilation.
Compilation web	Vérifier que l'application peut être transformée en une version web utilisable.
Tests de migrations	Vérifier que la base peut évoluer sans perdre sa cohérence.
Base de test séparée	Éviter que les essais modifient les données principales.
Transactions InnoDB	Annuler les données créées par un test afin que le test suivant commence proprement.
Vérifications de santé	Confirmer que le serveur démarre et répond correctement.

Lorsque les tests signalent une erreur, la modification est arrêtée, la cause est recherchée, puis la correction est vérifiée avant de continuer. Cette méthode réduit le risque de casser une partie déjà stable du projet.

3.4 Résultats obtenus

La version étudiée montre qu'il est possible de réunir plusieurs opérations d'une faculté dans une même architecture. Les principaux résultats sont résumés ci-dessous.

Domaine	Résultat
Comptes et rôles	La connexion, les rôles et les autorisations sont organisés côté serveur.
Demandes d'inscription	Les demandes peuvent être créées, consultées, approuvées ou rejetées.
Organisation académique	Les promotions, cours, étudiants, enseignants et inscriptions sont reliés.
Notes et résultats	Les évaluations, notes, moyennes, crédits et résultats officiels disposent d'une structure claire.
Valve et réclamations	Les annonces de cours et le suivi des problèmes académiques sont centralisés.
Enrôlements et projets	L'appariteur peut gérer les enrôlements et les affectations d'encadreurs.
Présences	Les séances, présences, refus et corrections peuvent être conservés.
Qualité technique	Le projet dispose de tests automatisés, de migrations et d'une compilation web vérifiée.
Biométrie	La fondation technique existe, mais l'usage facial complet reste une étape future.

Pour l'étudiant, le bénéfice principal est de retrouver plus facilement ses informations. Pour l'enseignant, le système offre un espace organisé pour ses cours et ses évaluations. Pour les responsables, il apporte une meilleure visibilité sur les opérations et conserve davantage de traces. Le projet reste toutefois un prototype avancé testé principalement en environnement local.

4. Difficultés rencontrées et limites

4.1 Difficultés rencontrées

Difficulté	Réponse apportée
Plusieurs technologies avaient été essayées	Le projet a d'abord connu des versions PHP et Flask. FastAPI a été retenu comme serveur actif et les anciennes versions ont été archivées.
Deux services MySQL pouvaient utiliser des ports différents	Le projet a été configuré pour utiliser le service WAMP prévu, tandis que les scripts vérifient la connexion avant les tests.
Les tests risquaient de toucher la vraie base	Une base séparée portant le nom smart_faculty_test a été rendue obligatoire pour les essais.
Certaines anciennes tables utilisaient un moteur peu adapté aux transactions	Les tables concernées ont été progressivement adaptées à InnoDB afin de mieux annuler les changements des tests.
Les règles académiques sont nombreuses et parfois sensibles	Les décisions non confirmées n'ont pas été inventées. Les calculs techniques ont été séparés des décisions officielles du jury.
Le projet est vaste pour une étudiante de L2	Le développement a été découpé en petits modules, avec une documentation et des tests après chaque étape.
La reconnaissance faciale demande des bibliothèques difficiles à installer	L'environnement Python, CMake et dlib a demandé plusieurs vérifications avant de pouvoir préparer cette partie.

Ces difficultés ont été utiles, car elles ont montré qu'un projet informatique ne consiste pas seulement à écrire du code. Il faut aussi organiser les outils, protéger les données, comprendre les règles et vérifier chaque décision.

4.2 Limites de la solution

Reconnaître les limites ne diminue pas la valeur du projet. Cela montre au contraire que le travail est présenté avec honnêteté.

Limite	Explication
Environnement local	La majorité des essais a été réalisée sur un ordinateur de développement. Un serveur réel peut présenter d'autres contraintes.
Utilisateurs réels	Des tests avec des étudiants, enseignants et agents de la faculté restent nécessaires pour mesurer la facilité d'utilisation.
Reconnaissance faciale	La structure est préparée, mais la caméra, l'anti-usurpation, la précision et les règles de protection des données doivent être validés.
Connexion Internet	Certaines fonctions dépendent de la disponibilité du serveur et du réseau.
Règles LMD	La progression annuelle, la seconde session et certains documents officiels doivent encore être complétés selon les règles validées par la faculté.
Projets académiques	Le suivi détaillé des réunions, livrables, commentaires et validations par étapes reste à développer.
Modules avancés	L'assistant académique, la carte du campus, les stages et les analyses avancées appartiennent surtout aux évolutions futures.
Sécurité de production	Un audit, le chiffrement, les sauvegardes, les journaux et les politiques d'accès doivent être renforcés avant un déploiement officiel.

La solution actuelle doit donc être considérée comme une base solide en développement. Elle montre la faisabilité du projet, mais ne doit pas encore remplacer sans préparation tous les procédés officiels de la faculté.

5. Conclusion générale

5.1 Récapitulatif

Smart Faculty a été conçu pour répondre à un problème simple à comprendre : les informations académiques sont souvent dispersées et difficiles à suivre. Le projet propose de les rassembler dans une plateforme destinée à une faculté. Flutter affiche les écrans, FastAPI contrôle les règles et MySQL conserve les informations.

La démarche a commencé par l'analyse des utilisateurs et de leurs besoins. Elle s'est poursuivie par la conception des données, l'organisation de l'architecture, le développement des modules et l'exécution de tests. La solution couvre déjà les comptes, les rôles, l'organisation académique, les notes, la Valve, les réclamations, les enrôlements, les projets et la base du suivi des présences.

5.2 Apports du projet

Ce travail a permis à l'étudiante de mieux comprendre la conception d'une base de données, la création d'un serveur, la communication entre une application et une API, la sécurité par rôles, les migrations, les tests et la documentation. Il a aussi développé une méthode de travail fondée sur de petites étapes vérifiables.

Pour une faculté, Smart Faculty pourrait réduire les recherches inutiles, limiter certaines erreurs, améliorer le suivi des demandes et rendre les informations plus accessibles aux personnes autorisées.

5.3 Perspectives

- Déployer la plateforme sur un serveur protégé avec des sauvegardes régulières.
- Faire tester les écrans par de vrais utilisateurs et simplifier les parties jugées difficiles.
- Finaliser la reconnaissance faciale avec consentement, anti-usurpation et mesure de précision.
- Compléter les règles LMD, la seconde session et les documents officiels.
- Ajouter les réunions, livrables et validations détaillées dans le suivi des projets.
- Développer progressivement l'assistant académique, la carte du campus, les stages et les tableaux de bord avancés.
- Poursuivre le projet en L3 afin de le rapprocher d'une utilisation réelle dans une faculté.

6. Bibliographie et Webographie

6.1 Bibliographie

Faculté des Sciences Informatiques, Université Protestante au Congo. Guide de rédaction de monographie, Licences 1 et 2 - LMD, 2024-2025.

Équipe Smart Faculty. Présentation, vision et périmètre du projet. Documents internes, 2026.

Équipe Smart Faculty. Analyse fonctionnelle, besoins et cas d'utilisation. Documents internes, 2026.

Équipe Smart Faculty. Architecture générale et architecture des interfaces de programmation. Documents internes, 2026.

Équipe Smart Faculty. Cahier des charges technique. Document interne, 2026.

Équipe Smart Faculty. Journal de développement et rapports de tests. Documents internes, 2026.

Équipe Smart Faculty. Conception de la base de données selon MERISE. Documents internes, 2026.

6.2 Webographie

Flutter, « Guide to app architecture », <https://docs.flutter.dev/app-architecture/guide>, consulté le 15 juillet 2026.

FastAPI, « Features », <https://fastapi.tiangolo.com/features/>, consulté le 15 juillet 2026.

Oracle, « MySQL 8.4 Reference Manual », <https://dev.mysql.com/doc/refman/8.4/en/>, consulté le 15 juillet 2026.

SQLAlchemy, « SQLAlchemy ORM Documentation », <https://docs.sqlalchemy.org/en/20/orm/>, consulté le 15 juillet 2026.

Alembic, « Documentation », <https://alembic.sqlalchemy.org/en/latest/>, consulté le 15 juillet 2026.

pytest, « Documentation », <https://docs.pytest.org/en/stable/>, consulté le 15 juillet 2026.

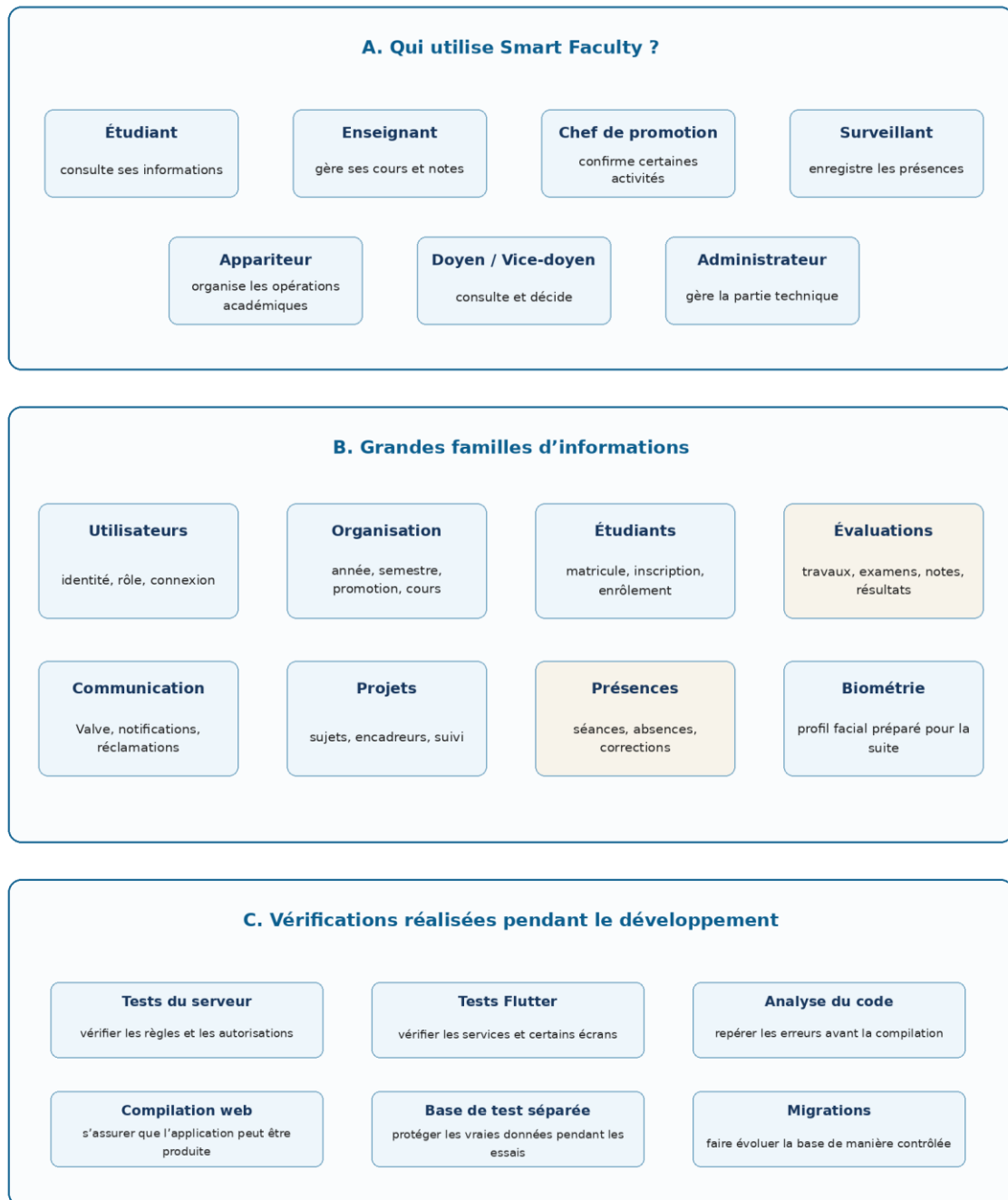
Python Software Foundation, « Python Documentation », <https://docs.python.org/3/>, consulté le 15 juillet 2026.

OWASP Foundation, « OWASP Top Ten », <https://owasp.org/www-project-top-ten/>, consulté le 15 juillet 2026.

7. Annexes

Les schémas ci-dessous résument le projet sans reprendre toutes les tables et tous les détails techniques. Ils servent à comprendre rapidement qui utilise la plateforme, quelles informations elle conserve et comment sa qualité est vérifiée.

Annexes visuelles simplifiées



Annexes A à C. Vue simplifiée des utilisateurs, des données et des contrôles de qualité.

Remarque : les modèles détaillés de la base, les extraits de code, les captures d'écran et les rapports complets de tests peuvent être ajoutés au dossier technique du projet lorsque la version destinée à la soutenance sera arrêtée.